

The AuditGH Screen

What we found, what we fixed, and the one thing we have not checked

11 September 2026 · Internal

Before anything else

This work is done.

- Not a proposal, not a plan
- Finished, in a branch, waiting to be merged
- Everything you are about to see was **measured**, not estimated

SPEAKING NOTES

Say this first. A deck about user interface quality is usually a pitch, and the room will listen for a budget ask. There is not one. Nothing needs to be purchased.

What this deck is asking for is a merge, an hour of pipeline configuration, and one to two weeks of one person's time on the follow-on. That is the whole ask.

What we are talking about

AuditGH is the screen our security team works from.

- Findings arrive, get sorted by severity, get assigned, get closed
- Every product like it has a **design system** — one agreed set of colours, written down once
- Like a brand manual: one navy, one grey, every slide uses the same ones

AuditGH did not have one.

SPEAKING NOTES

The brand manual analogy is the one to carry all the way through this deck. Do not swap it for another one halfway.

Nobody picks a shade of navy per business card. In AuditGH, colour was picked per screen, by whoever built that screen. That is not negligence — it is what happens when a product grows faster than anyone can stand back and look at it.

We counted the decisions

1,912

Individual colour choices, scattered across 148 files, instead of one agreed set.

SPEAKING NOTES

This is the number to let land. Pause after it.

It is not an estimate. A committed script counts it, and anyone in the room can run that script against the old code and get the same number back.

It also explains everything that follows. When colour is chosen in 1,912 places, you cannot test it, you cannot change it, and it will not agree with itself.

Seven findings

#	What we found	Count
1	“Low severity” shown in two different colours	5 screens
2	Colour instructions the browser silently discarded	36
3	Source files never in version control	5
4	Controls a screen reader could not identify	103
5	Colour pairs below the readability standard	5
6	Emoji used as interface icons	17
7	Layouts that could not fit a phone screen	15

SPEAKING NOTES

Three of these are the product literally not doing what the code says — findings 2, 3 and 5. Four are the product being harder to read and harder to trust than it should be.

Do not let the room merge these into one number. They have different consequences and they would have been found by different people at different times.

The three that are real defects, not polish

36 discarded colour instructions. Parts of the scheduling calendar were painting no colour at all. Broken silently, for as long as those lines existed.

5 files not in version control. A standard configuration rule borrowed from another programming language was skipping a folder of real source code. **A fresh copy of this project would not build.**

Faint outlines. The edge of every typing box measured 1.26 to 1 against a standard of 3. The keyboard focus ring measured 2.59.

SPEAKING NOTES

Lead with the version control one if the room is technical. It is the one that is a single wiped laptop away from being a real incident, and it is the one nobody was looking for — it fell out of measuring the baseline.

The calendar finding is the one that shows why this class of problem never gets reported. A highlight that never appears does not generate a ticket. People assume the feature does not exist.

103 controls a blind user could not identify

- **46 buttons** showing only a picture — a magnifying glass, a bin, an arrow
- A screen reader announced them as, literally, “button”
- **57 typing boxes** whose on-screen label was not connected to the box

All 103 now have names, taken from the label already printed beside them.

SPEAKING NOTES

This is the finding most likely to arrive as a customer requirement rather than as an internal priority. Public-sector and larger enterprise buyers increasingly ask for a formal accessibility statement.

Retrofitting after a customer asks costs more than doing it now, because by then it has a date attached and is a commitment rather than an engineering choice.

And it is not only customers. Our own colleagues who use assistive technology could not do this job on this screen.

What it looks like now

	Before	After
Individual colour choices	1,912	0
Discarded colour instructions	36	0
Emoji used as icons	17	0
Controls with no name	103	0
Layouts that cannot fit a phone	15	0
Total findings	2,284	0
Colour pairs testable	34 of 102	102 of 102
Colour pairs below standard	5	0

SPEAKING NOTES

Both columns come from the same script. That matters more than the numbers: change the ruler and both columns move together. This is not a before-picture drawn from memory.

The “34 of 102” line is the one worth explaining if asked. We could only test a third of the product’s colours before, because the other two thirds did not exist as named colours. You cannot test a palette that is not written down.

What it buys beyond the fixes

- **Changing a colour is one edit instead of 1,912.** All colour lives in one file.
- **It cannot quietly come undone.** Three committed scripts, runnable on every change.
- **Dark mode is now a design, not a guess.** All 88 colours defined in both themes.
- **The next screen is cheaper to build.** Four reusable building blocks added.
- **Colour is no longer the only signal.** Each severity has its own shape, so it survives a black-and-white print and a red-green colour vision deficiency.

SPEAKING NOTES

This is the slide that funds the follow-on. The fixes above are what we already paid for; this is what we keep getting.

The second bullet is the one to press. The difference between “we tidied it up” and “it stays tidy” is whether the checks run automatically. That is an hour of configuration and it is on the ask list.

The ask

Nothing to purchase. No licence, no vendor, no subscription.

Now — days:

1. Review and merge the branch
2. Turn the two checks on in the automated pipeline (about an hour)

Next — one to two weeks of one person:

1. Open the product in a browser at four screen widths and look at it
2. One pass with a screen reader over the main workflows

SPEAKING NOTES

Item 3 is the one that matters and the next slide explains why. Do not let it get dropped as a nice-to-have — it is the difference between what we have proved and what we have claimed.

Item 5 on the full list is a decision rather than a task: what to do about exported PDFs and spreadsheets, which carry their own appearance when they leave the product. It does not need a project. It needs somebody to choose.

Then we opened it, and it broke twice

Our checks said zero. A browser said otherwise, in under a minute.

- **Navigation menu** — three groups accidentally sharing one internal name
- **Dashboard radar graphic** — 4 places stopped with an error, **9 more silently painted the wrong colour**

Both fixed. The second is now covered by a **13th automatic check** — 27 problems against the broken version, 0 against the fixed one.

SPEAKING NOTES

Do not skip this slide because it is unflattering. It is the most persuasive slide in the deck for the ask on the previous one.

The nine silent ones are the point worth making aloud. An error that stops is cheap — you see it. A wrong colour that carries on is the expensive kind, and it is exactly the kind our static checks are worst at finding.

If someone asks why we did not catch it: a type check and a production build both passed too. This class of fault only exists at the moment the page runs.

What we are not claiming

The measurement work was done without a browser.

Everything was measured by reading the source code, not by running the application.

- We **can** prove no instruction in the code causes a page to be too wide for a phone
- We **cannot** prove no page is too wide for a phone

Those are different statements. “Zero findings” means **zero findings our checks can see** — not zero defects.

SPEAKING NOTES

End here, deliberately. If the room takes one caveat away, make it this one.

Reading the source covers all 148 files exhaustively and reproducibly. Clicking through the application covers whatever a person remembers to click. We chose the first, and it has a precise blind spot, and item 3 on the previous slide is how we close it.

Also say: nothing here is blocked by access or permission. There is no system we cannot reach and no privilege we do not hold. It is one person, a browser, and a week or two.

Where the detail is

- **Technical companion:** `ui-design-system-report.md` — every measurement, every mechanism
- **Named appendix:** `ui-design-system-appendix.md` — every location, generated by script
- **Reproduce it yourself:** `./scripts/report/ui-baseline-compare.sh`

SPEAKING NOTES

Offer the last line to anyone who wants to check the numbers rather than trust them. It runs in under a minute and prints both columns of the before/after table.